Figaro

A proof of delivery blockchain solution to the last mile problem

Fabrizio Romano Genovese

fabrizio.eth

March 30, 2022

Contents

Contributors **2**
Acknowledgements . . . 2

1 Introduction **3**
1.1 The last mile problem . . . 3
1.2 Last mile problem solutions leveraging on community . . . 4
1.3 Figaro: A more ethical solution . . . 4
1.4 Prerequisites . . . 5
1.5 Synopsis . . . 5

2 Protocol Overview **6**
2.1 Impossibility of Proof Of Delivery . . . 7
2.1.1 Proof of Action . . . 7
2.2 Choosing consensus protocol . . . 8

3 Types of transactions **9**
3.1 Request transaction . . . 10
3.2 Cancel request transaction . . . 12
3.3 Take transaction . . . 13
3.4 Cancel take transaction . . . 15
3.5 Pick-up transaction . . . 17
3.6 Delivery transaction . . . 17
3.7 Slashing transactions . . . 19

4 Protocol execution **20**

Bibliography **21**

Contributors

This work is mainly the result of months of private conversation between F. R. Genovese and A. Daliana. We were both animated by the desire of seeing blockchain evolve to something going beyond finance, and looked with interest at the development of Decentralized Anonymous Organizations (DAOs) and governance systems. F.R.Genovese has a technical background, while A.Daliana has many years of expertise in managing logistic systems. Logistics is a stricly producion-focused, product-oriented business and so it seemed like a natural starting point to work on.

The idea of conceptually substituing PoW with PoS was Alessandro's. Fabrizio found a technical way to reify it.

The majority of this document has been written by F.R.Genovese, with contributions by the following people:

Acknowledgements

We would like to thank the crypto ecosystem as a whole and the Ehtereum community in particular, that never stops to provide amazing insights and food for thought.

We also want to thank Vlad Zamfir, without which we would have never met.

Chapter 1

Introduction

Figaro is a blockchain-based solution to the last mile problem [13] in logistics. In Proof of Work (PoW) blockchains such as Bitcoin [10], the right of minting a block depends on spending computational power to solve a difficult mathematical problem such as inverting a hash [1]. Providing such a solution testifies that a certain amount of energy has been spent in doing some work. Figaro starts from the very simple conceptual assumption that, in logistics, a proof that a given good has been delivered from some point A to some point B provides evidence of work, and can be used to earn the right to mint a block. We call this idea *Proof of Action (PoA)*.

In this work, we employ PoA to design a community-based, decentralized, autonomous solution for the last mile problem.

The name *Figaro* plays hommage to the main character of Gioacchino Rossini's *Il Barbiere di Siviglia*. Figaro is the city *factotum*, running around making errands here and there for a diversified plethora of clients.

1.1 The last mile problem

In the field of logistics, the wording *last mile problem* refers to the last leg of a supply chain. It involves transportation of goods from a hub of some sort to its final destination, usually the consumer's home.

Last mile has its own sort of peculiarities that set it apart from other stages in the logistics chain. Importantly, in moving products from transportation hub to transportation hub said products can be bundled together, resulting in a lower shipping cost per item.

Example 1.1.1 (Container ships)**.** A container ship costs a lot of money to move, but it can transport many tons of goods. For instance, in shipping a container from one side of the Atlantic to the other, the shipping cost one would pay would spread proportionally across all the items in the container. Shipping many containers every month further lowers expense per item as one would benefit from many *bulk shipping* pricing options.

This is not necessarily true for the last mile problem, as final consumer locations are scattered around and the storage capacity of the involved means of transportation – usually small trucks, cars, mopeds and even bikes – are limited.

Example 1.1.2 (Pizza delivery)**.** If a manager of a Pizza restaurant needs to deliver pizza to their clients, they will rely on a delivery service of some sort, such as a rider driving a moped. One can moreover imagine that, on average, every client orders just two or three pizzas. In this case many

trips will have to be scheduled: the capacity of transporting many pizzas at once is hindered by the fact that

- Pizza will have to be delivered hot;
- Storage space is limited.

As such, the price per shipment for pizza delivery will probably be *higher* than the cost of shipping one unit of weight of flour from the producer to the pizza restaurant, as in Example 1.1.1.

Another big issue of last-mile logistics is that end consumers are on average less reliable than big sorting hubs. Deliveries have often to be rescheduled because the customer is not at home. Moreover, there is an high incidence of theft, especially when packages are left at the door [8].

The peculiarities of the last mile problem have been increasingly studied in the last years, thanks to the boom of shared mobility and business-to-consumer activities such as e-commerce. What makes it a difficult problem to solve is how it varies from area to area – compare for instance a huge north-American metropolis with regular, wide streets with a historic small European town where streets are narrow – and from product to product – pizzas as in Example 1.1.2 have to be delivered as soon as possible, while other goods such as consumer electronics can take longer.

1.2 Last mile problem solutions leveraging on community

In the last decade or so, with the advent of mobile technology, a new class of solutions to the last mile problem have come to light. The main idea is that in a sufficiently densely populated city the inhabitants may be interested in providing delivery services to round up their salaries. If this turns out to be remunerative enough, they could consider making it a full time job. This is the core working model of companies such as Uber [15] and Lift [14].

Indeed, almost anyone with a car can become a Uber driver and earn money by providing mobility services to passengers. This is made possible by the fact that almost anyone owns a smartphone now, which allows to monitor and manage very granularly the flow of operations. Similarly, one can earn money by delivering food from restaurants to consumer houses. This happened for decades, but the presence of apps coordinating pickups from multiple locations free the workers from being bound to one particular restaurant about as much as they free restaurants from having to manage the delivery process in-house.

Unfortunately, the advent of these app-based resulted in an overall very exploitative business, for multiple reasons:

- First of all, these delivery systems rely on metrics like speed of delivery and customer reviews. As it is for services such as TripAdvisor [11], the impact of negative reviews weights far more than the impact of positive ones. This results in a lot of undesirable practices such as food delivery operators speeding [5], putting theirs and everyone else's safety at risk in the hope of avoiding bad feedback.
- Secondly, apps such as Uber have huge running costs, both related to the infrastructure they have to maintain and to the need of maximizing profit to satisfy their shareholders. This results in the company managing the app retaining the majority of the shipment costs, sometimes over 70% [].

1.3 Figaro: A more ethical solution

All things considered, as it often happens an important problem has been partially solved at the expense of the poorest and the weakest. The underlying idea of Figaro is about decentralizing the

whole last mile logistics. People making deliveries coordinate through a decentralized, blockchain based network. External costs are reduced at a minimum as the ones making deliveries are themselves the miners: PoW is replaced by *Proof of Action (PoA)*, that is, a cryptographically secure proof that an item has been safely delivered from a point A to a point B. As such, deliverers are themselves the miners, and mining profits – that is, delivery fees – are almost entirely redistributed to the workers. Moreover, on-chain data makes it possible to run a reputation-based system that allows workers to participate into a governance system.

Long story short, Figaro is a hybrid between a blockchain protocol and a workers' union.

1.4 Prerequisites

Notation 1.4.1. We will use the word *miner* whenever we refer to an agent minting blocks in a Proof of Work blockchain, and the word *stacker* whenever we refer to an agent minting blocks in a Proof of Stake [] blockchain. Finally, we will use the word *rider* whenever we refer to an agent minting blocks in a Proof of Delivery blockchain.

This document will be technical, but not overly mathematical. A general knowledge of how blokchain technology works is required, together with basic knowledge of cryptography, such as what is cryptographic signing or hashing. Troughout the paper, overly technical/mathematical remarks will be highlighted with an asterism, as in:

Remark* 1.4.2 (An overly mathematical remark from Wikipedia)**.** «To create signature keys, generate an RSA key pair containing a modulus, N, that is the product of two random secret distinct large primes, along with integers, e and d, such that $ed \simeq 1(\mod \phi(N))$, where ϕ is the Euler's totient function. The signer's public key consists of N and e, and the signer's secret key contains d.

To sign a message, m, the signer computes a signature, σ, such that $\sigma \simeq md(\mod N)$. To verify, the receiver checks that $\sigma e \simeq m(\mod N)$.»

We will do our best to shape this work in such a way that remarks as Remark* 1.4.2 can be safely skipped without hindering general comprehension.

1.5 Synopsis

- In chapter 2 we give a basic overview of the protocol;
- In chapter 3 we list the various different types of transactions in the Figaro protocol, and explain their roles in detail.

Chapter 2

Protocol Overview

As we already stated in chapter 1, we envision Figaro as a soluton to the last-mile problem, that works by organizing deliveries between clients in a decentralized way.

Remark 2.0.1 (Actors involved)**.** Here are the main actors in the protocol:

Clients: these are people that need to send or receive parcels. They send out delivery requests in the form of transactions, specifying how much they are willing to pay for their delivery;

Riders: these are workers that deliver the goods from point A to point B, in exchange of the delivery fee set by the client. They effectively mine transactions.

These actors can either be

Honest: meaning that they strictly abide to protocol rules;

Rational: meaning that they will prefer one legal protocol state more than another if this results in a better personal return;

Malicious: meaning that they try to game the protocol to favor a particular outcome (such as double-spending a transaction).

Remark 2.0.2 (Working context)**.** There are moreover other peculiarities to account for, which depend on the unique nature of logistic-related processes. Indeed, our protocol will be:

Geofenced: as we deal with last mile deliveries, we can imagine clients and riders as living in a restricted geographic area, such as a town or city. This also creates time-related differences in the protocol execution, as activity will be maximally concentrated in precise timeframes (such as lunch and dinner time for food delivery).

Non-financial: we are not interested in payments but in deliveries. As such, the only allowed forms of payments should be deposits, withdrawals and delivery fees. This is crucial to avoid exploitation of the chain for means other than deliveries.

Price capped: even in the context of last-mile logistics, deliveries are not all created equal. For instance, food delivery usually involves low fees, and the value of the goods trasported is itself modest; on the other hand of the spectrum we have security carriers, such as the ones delivering cash to banks. In this paper, we focus on the former, for several reasons:

- Smaller deliveries are more abundant in number;

- They require less bureaucracy – e.g. delivering cash to banks is arguably beaurocratically more difficult than delivering pizzas;
- They don't require workers to own or rent specialized tools for the job – e.g. an armored truck versus a moped.

These characteristics provide an large, easy to recruit rider crowd to run the protocol.

2.1 Impossibility of Proof Of Delivery

The most obvious idea to implement a decentralized solution for logistics would be to tie blockchain consensus to *Proof of Delivery*. In short, proving that a given package has been delivered from A to B should give the rider the right to mint a block.

The problem with this is that riders and clients can collude, often in catastrophic ways. For instance, the same malicious actor could impersonate both a couple of clients and a rider, and produce fake deliveries between them where no package exists and no delivery has really taken place. In tying consensus – and thus block minting – to deliveries, this would effectively translate in the malicious actor being able to completely manipulate the protocol.

A solution to this would be resorting to some cryptographic primitive to prove that a given actor is in some well-defined geographical location. In this way, a rider would be required to provide cryptographic proofs of being first in location A and then in location B. Even if the rider and the clients in A and B collude, now we are sure that some work – that is, the rider moving from A to B – has taken place.

Unfortunately, this is not possible. In [4] authors proved an impossibility theorem establishing how no such classical cryptographic protocol is possible without making strong assumptions on the actors involved.[1] Moreover, said protocols require direct, speed-of-light communication between devices (usually through radio waves) and/or a trusted setup of verifiers. On the other hand, we require our protocol to be run on a common smartphone with nothing more than a internet connection and no direct radio communication between riders' devices.

2.1.1 Proof of Action

Given that proof of delivery is not cryptographically feasible, we resort to the next best thing, which we call *Proof of Action (PoA)*. The underlying idea is that even if we cannot faithfully represent some action in cryptographic terms, we can assume that said action took time to be performed: time is much easier to represent and in this paper we give two ways of doing so, one more secure but more computationally expensive, and the other less secure but computationally cheap.

Clearly, we are now stuck with the same problem we had before, that is, colluding parties can represent actions that did not really take place. They will have to wait a certain time to do so, but will still have an advantage over honest riders having to put in real work to perform the action. Indeed, fake riders won't sleep or have any expenses such as fuel, and will be able to produce fake transactions all day long.

This apparently useless addition can save the day if we notice something else: actions such as shipments are made in at least two steps, the pick-up and the delivery. Schemes that minimize trust work via interlocking escrows: a client escrows the shipping fee, whereas the rider escrows some funds at pick-up time to insure the value of the shipment. These funds are unlocked after

[1] Interestingly, this result does not hold anymore if we postulate that actors can share quantum resources [2]. This may become relevant in the future as quantum technology improves, but it is outside of the scope of this work for now.

the final delivery is confirmed. All sorts of failures translate to escrows being slashed or forfeited for one or both parties.

In this setup, a client may be trying to double-spend a transaction, but clearly it makes sense to do so only after delivery is confirmed, as a roll-back before pick-up would result in the rider not showing up at all, whereas a roll-back between pick-up and delivery would result in the escrowed sum being backrolled and the rider being effectively able to run away with the cargo.

As such, if we have a system in place to provide finality somewhere in time between shipment request and delivery, the double-spending problem would trivially vanish: as request becomes final, the most rational outcome for the client is to play along with the rules and go through with the shipment lifecycle. These considerations will be further analyzed in [].

2.2 Choosing consensus protocol

In light of these considerations, we conclude that the simplest and smartest way to implement PoA is to rely on some other *finality provider*, or to employ it in the context of a consensus protocol with finality. There are multiple options for this, that will be discussed further in []. Whatever consensus mechanism we chose, these are the properties we need:

Finality time shorter than average delivery time: this effectively takes out and incentive to double-spend in light of the considerations made above. In chapter 3 we will cover the transaction structure that effectively implements our locked escrow system.

Timestamping: time is really important for us, as shipments – especially in the field of food deliveries – need to be precisely timed ('no one wants to receive a cold pizza'). Again we have many options to represent time, at least in approximate terms, in a blockchain context. See [] for details.

Representing stablecoins: in contrast with other financially-focused blockchain projects, Figaro relies on a logic of production, where clients use it to deliver goods and riders use it to earn their daily living. It is apparent how Figaro basic currency must be as little volatile as possible to ensure the price stability of riders' earnings. A lot of research has already been made and implemented in the field of stablecoin cryptocurrencies, with alternatives ranging from centralized solutions [3] to algoritmic ones [9], to a mix of the two [6].

Remark 2.2.1 (Figaro as a boost for stablecoin innovation)**.** The geographically local nature of Figaro may provide a boost for the development of stablecoins based on currencies different from USD. Euro [] and GPB [] based stablecoins already exist, but have not seen much adoption since the financial system is largely dollar-centric. On the contrary, it is reasonable to assume that workers working in a particular country want to be paid in that country's native currency. This will most likely include Figaro riders.

Chapter 3

Types of transactions

Whereas financial operations, such as sending money, can be considered *atomic* – they happen in one go – logistics is done in *stages*: a request for shipment has to be put out, and the shipment must be assigned to an intermediary. Furthermore, both pick up and delivery confirmations are needed. In a decentralized context we want to minimize trust as much as possible, and scrapping out one of these steps immediately results in a bad system of incentives, such as the rider being able to steal the merchandise.

For these reasons, Figaro has different types of transactions, that give rise to an *interlocked escrow system*. Whereas some transactions can be called 'as-is', other transactions need to refer to previous transactions to be considered valid. The dependence graph between different types of transactions is shown in Figure 3.1, where the arrow means 'must refer to'.

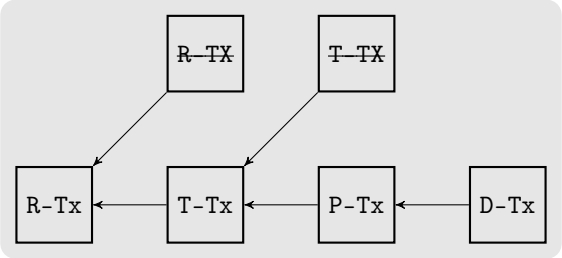


Figure 3.1: Basic Figaro transactions and their dependencies: `R-Tx`, request transaction, section 3.1; ~~`R-TX`~~, cancel request transaction, section 3.2; `T-Tx`, take transaction, section 3.3; ~~`T-TX`~~, cancel take transaction, section 3.4; `P-Tx`, pick-up transaction, section 3.5; `D-Tx`, delivery transaction, section 3.6;

Notation 3.0.1 (Dramatis personæ)**.** In this chapter, we will refer to the various actors involved as follows:

Sender: the individual shipping some good;

Recipient: the individual receiving some good;

Rider: the individual taking charge of the delivery;

Signee: whoever cryptographically signs a transaction.

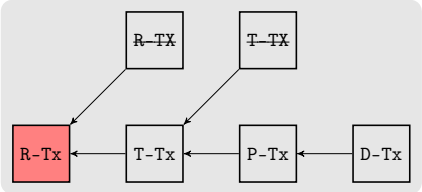
We will often refer to these actors as they were proper names, as in 'rider does this, then sender does that'.

We will moreover use *object notation* to specify nested properties: If A has some subproperty B, we will write $A.B$ to denote 'property B of A'. In particular, we will write $A.\texttt{signee}$ to denote the wallet that cryptographically signed A.

Remark 3.0.2 (Counting time)**.** As we remarked in section 2.2, we need a notion of timestamping to formalize many of the concepts we need. Abstracting from the particular consensus protocol we chose, we denote the time of any transaction in *ticks*. This is an integer that can represent the block number in which the transaction is, an actual timestamp or something else, depending on the consensus mechanism chosen.

Finally, we will treat time as a property, hence we will write $A.\texttt{Time}$ to denote the time of A.

3.1 Request transaction



Request transactions (`R-Tx`) represent requests for a shipment. These transactions can be sent by either sender or recipient, and signal demand for a shipment. They contain a monetary value – how much one is willing to pay rider for the service, vague location estimates to allow potential riders to evaluate the offer, and an indication of the value of the shipment, that rider will have to escrow for security reasons.

Definition 3.1.1 (Request transaction)**.** *A* request transaction*, also denoted `R-Tx`, is composed of the following fields:*

`from`**:** *the wallet requesting the shipment;*

`to`**:** *the wallet of the final recipient;*

`fromArea`**:** *a integer. Represents the GPS coordinates of the pick-up address, expressed in* decimal grades *[12];*

`toArea`**:** *a integer. Represents the GPS coordinates of the delivery address, expressed in* decimal grades *[12];*

`insurance`**:** *a positive integer. The value of the package to be shipped;*

`fee`**:** *a positve integer. What the requesting user is willing to pay for the shipment;*

`appointmentTime`**:** *a positive integer. The pick-up time for the rider, expressed in* ticks *(see Remark 3.0.2), giving indication to the rider for a time to show up;*

`noShow`**:** *a positive integer. A further time interval, expressed in* ticks *(see Remark 3.0.2), determining the latest time at which the rider can show up;*

`nonce`**:** *a positive integer, ensuring that the same transaction is not repeated.*

To be valid, the transaction must satisfy the following conditions:

- *`from` and `to` must be valid wallets;*

- *`fromArea` and `toArea` must lie within a geofence established at protocol level (see-[]);*
- *It must be `insurance` $\leq$ `maxInsurance`, where `maxInsurance` is a value established at protocol level;*
- *It must be `baseFee` $\leq$ `fee` $\leq$ `maxFee`, where both `baseFee` and `maxFee` are estblished at protocol level;*
- *It must be `lastNonce` $\leq$ `nonce`, where `lastNonce` is the nonce of the last transaction signed by `signee`. `lastNonce` is considered to be* 0 *if no such transaction exists;*
- *`signee` must either be `from` or `to`;*
- *`signee` must have at least `fee` in their wallet.*

The transaction has the following effects:

- *Locks `fee` in escrow from whoever signed the transaction.*

Remark 3.1.2 (Both `from` and `to` can initiate a transaction)**.** The reason why we require a `R-Tx` in Definition 3.1.1 to be signed either by `from` or `to` is for fiscal reasons.

In standard usage (see Remark 3.1.3 for non-standard usage) `from` is the sender, and `to` is the recipient. Traditionally, the delivery fee is paid by sender to the rider, and the recipient receives the package for free. But if sender is an acivity such as a restaurant, the payed delivery fee will actually be charged to recipient at the time of purchase. As one can see, the money flow is

$$\text{sender} \to \text{recipient} \to \text{rider}$$

and, especially if sender has a lot of orders, these delivery fees will pile up, resulting in additional bureaucratic hassle. Moreover, the money flow sketched above may not be innocuous tax-wise. This happens everytime sender is in a fiscal regime that does not allow to declare expenses, as it happens for the Italian "regime forfettario" [] directed at small businesses.

Allowing recipient – in our case `to`– to sign the transaction creates a simplified logic where sender only receives the payment for the goods sold, and does not have to mind about accounting shipment costs.

Moreover, it creates less incentives for double-spending, as shipments departing from a seller are arguably many more than shipments directed at a particular client. As `fee` can be completely offloaded to recipient, they are more evenly spread out, creating less 'big spenders' with a high incentive to rollback.

Remark 3.1.3 (`from` and `to` can be the same)**.** As it will become clear in the subsequent sections, the role of `from` and `to` in `R-Tx` is mainly acknowledging that a shipment has been picked up and correctly delivered, respectively. Allowing `from` and `to` to coincide models a *non-standard usage* situation, where both `from` and `to` coincide either with sender or with recipient. This may be useful in situations where one between sender and recipient does not use Figaro.

Example 3.1.4 (An example of non-standard usage)**.** Suppose sender and recipient are good friends living in the same town. Suppose moreover that sender baked some cake and wants to deliver a slice to recipient. If sender uses Figaro and recipient does not, sender can specify their own wallet as both `from` and `to`, and set `fromArea` and `toArea` as sender and recipient respective addresses.

When recipient receives the cake, recipient will call sender, that will approve the delivery and release any outstanding escrows (this can be further automated at UI level).

Arguably, this scenario is more inconvenient for the rider: at delivery time, any arguments should be directed to sender, that is not there. But then again, UI can notice rider that sender and recipient are the same, so that rider can take on an informed risk.

Remark 3.1.5 (The role of `fromArea` and `toArea`)**.** The parameters `fromArea` and `toArea` are needed so that riders can calculate an approximate delivery path and estimate if the provided `fee` is good enough for them. `fromArea` and `toArea` are approximative locations to protect clients' privacy, and are requested to be accurate only within a radius of n meters, where n is fixed at protocol level. The exact pick-up and delivery locations are disclosed to the rider at a latter stage, see Remark 3.3.3.

Remark 3.1.6 (The role of `insurance`)**.** `insurance` is a value estimation of the goods being delivered, provided by the client issuing the transaction. Is is used in a escrow logic to ensure delivery, see section 3.4.

Remark 3.1.7 (The role of `fee`)**.** `fee` is a value estimation of the goods being delivered, provided by the client issuing the transaction. `fee` can be further decomposed as `baseFee` + `deliveryFee`. As remarked in Definition 3.1.1, `baseFee` is fixed at protocol level and represents the 'price to pay' to use the protocol. We are interested in making this value as small as possible to incentivate use. On the other hand `deliveryFee` is decided by whoever signed the transaction. We expect a market to form around `deliveryFee` similar to the gas market for Ethereum, with prices depending on distance, location and congestion.

Remark 3.1.8 (`insurance` and `fee` are capped)**.** `insurance` and `fee` are capped at protocol level by `maxInsurance` and `maxFee`, respectively. The main reason for this is to disincentivate protocol abuse.

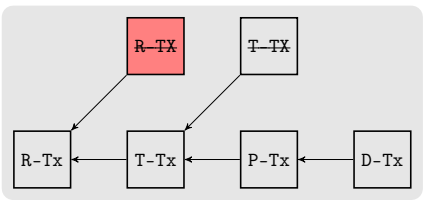
The idea is that riders can decide together how much a single delivery should cost in their area. If we are talking about food deliveries in a small city, it may be reasonable to assume that a single delivery should not exceed 10$.

Similarly, the insurance capping serves the purpose of implicitly selecting a target audience for the delivery system. Again, in the case of food delivery, it could be reasonable to assume – depending on the geographic area – that the insurance value does not exceed, say, 200$.

Remark 3.1.9 (The role of `appointmentTime` and `noShow`)**.** The parameters `appointmentTime` and `noShow` provide two deadlines for the rider. `appointmentTime` has no role in the protocol. Is just an indication of when, more or less, the rider will have to show up at sender's location to pick-up the package.

`noShow` is the maximum time that sender can wait. If rider does not show up by then, it will cause a problem to sender, as in the case of some hot food ready for delivery getting cold. `noShow` has a role at protocol level, see section 3.4 for details.

3.2 Cancel request transaction



Cancel request transactions (~~R-TX~~) represent order cancellations. They can be used before an order is picked up by a rider. They signal a change of mind in either the shipping or the receiving client.

Definition 3.2.1 (Cancel request transaction)**.** *A* cancel request transaction*, also denoted ~~R-TX~~, is composed of the following fields:*

`R-Tx`**:** *the hash of a* `R-Tx`*;*

`nonce`**:** *a positive integer, ensuring that the same transaction is not repeated.*

To be valid, the transaction must satisfy the following conditions:

- `R-Tx` *must be the hash of a valid* `R-Tx` *transaction;*
- *No other valid transaction must point to such* `R-Tx`*;*
- *It must be* $\texttt{lastNonce} \leq \texttt{nonce}$*, where* `lastNonce` *is the nonce of the last transaction signed by* `signee`*.* `lastNonce` *is considered to be* 0 *if no such transaction exists;*
- `signee` *must either be* `R-Tx.from` *or* `R-Tx.to`*.*

The transaction has the following effects:

- *Slashes a quantity* `baseFee`*, fixed at protocol level, from the escrow of* `R-Tx.signee`*;*
- *Returns the remaining* `deliveryFee` *(cfr. Remark 3.1.7) to* `R-Tx.signee`*;*

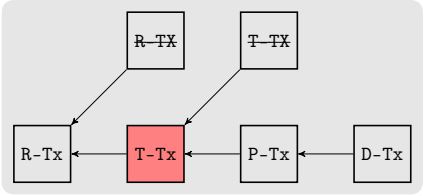
Remark 3.2.2 (Both `from` and `to` can cancel the transaction)**.** Remarkably, both `from` and `to` can cancel a transaction. Looking at Definition 3.2.1, this means that potentially `from` is able to cause a financial loss to `to`, and vice-versa. We do not deem this to be a problem. Indeed, we give for granted that an agreement between `from` and `to` already exists at the moment of creating a `R-Tx`. Should this not be the case, for any reason, any further conflict could be dealt between `from` and `to` separately.

Example 3.2.3 (`from` cancels a `to` transaction)**.** Suppose that `from` is a restaurant, and that `to` is a client. `to` orders some dumplings using a food delivery app relying on Figaro as a delivery backend.

Suppose moreover that, very soon after `to` ordered, `from` realizes that the restaurant has ran out of dumplings. `from` quickly cancels the transaction, that wasn't yet picked up by a rider. In doing so, `from` causes a financial loss to `to`. Still, `from` needs to repay `to` of the money spent. `from` will add `baseFee` to the refund.

Remark 3.2.4 (No-pointing condition)**.** In Definition 3.2.1 we require that no other transaction must be pointing to `R-Tx`. This is to avoid cancellation of a `R-Tx` that is either already canceled or already taken. A very similar condition will be required also in Definition 3.3.1, Definition 3.4.1 and Definition 3.5.1.

3.3 Take transaction



Take transactions (`T-Tx`) represent a rider committing to take charge of a given shipment. In doing so, the rider commits money in escrow, that will be eventually unlocked when the shipment takes place (see section 3.5).

Definition 3.3.1 (Take transaction)**.** *A* take transaction*, also denoted `T-Tx`, is composed of the following fields:*

`R-Tx`**:** *the hash of a `R-Tx`;*

`nonce`**:** *a positive integer, ensuring that the same transaction is not repeated.*

To be valid, the transaction must satisfy the following conditions:

- *`R-Tx` must be the hash of a valid `R-Tx` transaction;*
- *No other transaction must point to `R-Tx`;*
- *It must be `lastNonce` $\leq$ `nonce`, where `lastNonce` is the nonce of the last transaction signed by `signee`. `lastNonce` is considered to be* 0 *if no such transaction exists;*
- *`signee` must have at least `R-Tx.insurance` + `baseFee` in their wallet.*

The transaction has the following effects:

- *Locks a value `R-Tx.insurance` + `baseFee` in escrow from `signee`.*

Remark 3.3.2 (Escrow value is big)**.** The size of the esrow is `R-Tx.insurance` + `baseFee`. This covers two different liabilities:

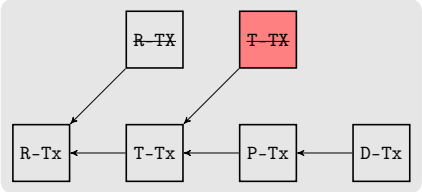
- First of all, `signee`– that is, the rider – must have at least `R-Tx.insurance` in her wallet, which is the needed minimum to ensure the shipment won't be lost or stolen. Even if rider is not yet picking up the goods, it makes no sense allowing her to do so if she cannot guarantee for the value of the merchandise;
- Secondly, if rider does not show up she will have to refund the issuer of `R-Tx` of the protocol fee used, see section 3.4 for details.

Remark 3.3.3 (Discosing full information)**.** Now that rider has taken charge of the delivery, she must know the precise locations of pick-up and delivery – remember that for privacy reasons `fromArea` and `toArea` are approximative locations within a radius of n meters from the real ones, see Remark 3.1.5.

There are various ways to implement this process, but we find the most elegant to be this: as for [], a Figaro wallet is nothing more than a public and a private keypair. Keypairs can be used not just for signing, but also for encrypting. At this point, rider knows the public keys of sender and recipient – they were on the `R-Tx` to which the `T-Tx` points, and dualizing this logic the inverse also holds. This means that sender, recipient and rider can exchange encrypted messages between each other. A clever frontend implementation of Figaro would use this function to implement a messaging protocol on top of the shipment layer, with which relevant information can be exchange (maybe even in an automated way).

Example 3.3.4 (Building trails of evidence for refunds)**.** If we stipulate that all messages are first signed and then encrypted, these could also be used as unforgeable evidence in a governance system built on top of Figaro. For instance, if sender refuses to disclose the pick-up location to rider, forcing rider to cancel the ride (see Definition 3.4.1), any messaging evidence could be taken to the governance layer to ask for a refund.

3.4 Cancel take transaction



Cancel take transactions (~~T-TX~~) represent a rider that, for whatever reason, has to forfeit an order already taken. In doing so, the rider is punished by losing a part of her own escrow.

Definition 3.4.1 (Cancel take transaction)**.** *A* cancel take transaction*, also denoted ~~T-TX~~, is composed of the following fields:*

`T-Tx`**:** *the hash of a* `T-Tx`*;*

`nonce`**:** *a positive integer, ensuring that the same transaction is not repeated.*

To be valid, the transaction must satisfy the following conditions:

- `T-Tx` *must be the hash of a valid* `T-Tx` *transaction;*
- *No other transaction must point to* `T-Tx`*;*
- *It must be* $\texttt{lastNonce} \leq \texttt{nonce}$*, where* `lastNonce` *is the nonce of the last transaction signed by* `signee`*.* `lastNonce` *is considered to be* 0 *if no such transaction exists;*
- `signee` *must be* `T-Tx.signee`*.*

The transaction has the following effects:

- *Returns* `T-Tx.R-Tx.fee` *to* `T-Tx.R-Tx.signee`
- *Slashes* `baseFee` *from* `signee` *escrow;*
- *Sends to* `T-Tx.R-Tx.signee` *a quantity* x *of the* `signee` *escrow, calculated using the formula:*

$$x = \begin{cases} \left\lfloor \frac{\texttt{noShowPercentage} \cdot \texttt{T-Tx.R-Tx.insurance}}{\Delta} \right\rfloor \cdot (\texttt{Time} - \texttt{T-Tx.Time}) & \text{if } \texttt{Time} < \texttt{R-Tx.Time} + \Delta \\ \texttt{noShowPercentage} \cdot \texttt{T-Tx.R-Tx.insurance} & \text{if } \texttt{Time} \geq \texttt{R-Tx.Time} + \Delta \end{cases} \tag{3.1}$$

  Where $\Delta = \texttt{T-Tx.R-Tx.appointmentTime} + \texttt{T-Tx.R-Tx.noShow}$*,* `Time` *is the current time in ticks and* `noShowPercentage` *is fixed at protocol level;*

- *Returns the remaining quantity of* `signee` *escrow to* `signee`*.*

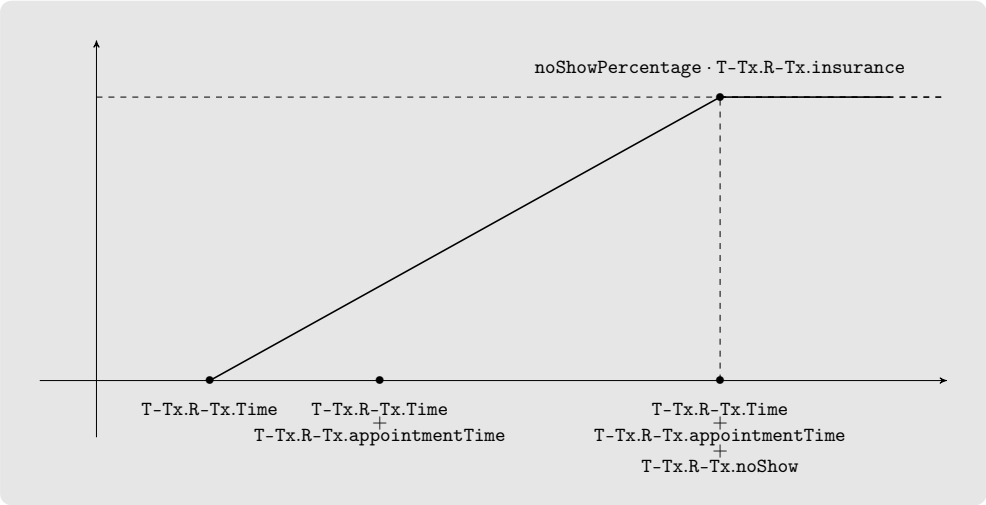
Remark 3.4.2 (Original issuer gets refunded)**.** The most striking feature of Definition 3.4.1 is that `T-Tx.R-Tx.signee` gets its fee back. This is normal as the service was never properly given, and receiving a refund may motivate `T-Tx.R-Tx.signee` to replace the old `R-Tx` with a new one.

Remark 3.4.3 (Rider pays for protocol expenses)**.** Moreover, in Definition 3.4.1 rider escrows a quantity `R-Tx.insurance`+`baseFee`. This is because as `fee` is fully refunded to `T-Tx.R-Tx.signee`, this refund includes the `baseFee` needed to run the protocol. So not only must rider refund the dissatisfied client, but also the protocol itself to cover costs.

Remark 3.4.4 (Rider partially refunds `insurance` value)**.** In addition to this, rider refunds the `T-Tx.R-Tx.signee` with a quantity ranging from 0 to a fixed fraction `noShowPercentage` of `R-Tx.insurance`, which increases with time. This is because there is reason to believe that the inconvenience caused will grow as the deadline for pick-up approaches – imagine for instance a pizza restaurant waiting in vain for a rider, until pizzas get cold.

This refund scheme also incentivates the rider to forfait a taken transaction as soon as possible to minimize loss, giving more time to the original issuer to reorganize.

Remark* 3.4.5 (Calculating proportional refunds)**.** The apparently complicated formula in Definition 3.4.1 draws the plot of the following function, where the x-axis is time and the y-axis is the slashing size:



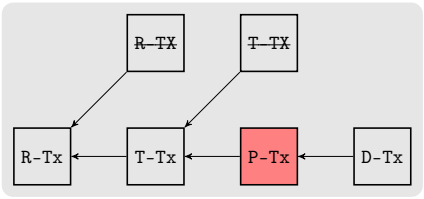
As one can see, the cost of cancelling a `T-Tx` grows linearly as the 'no-show' deadline approaches. In this setup it is evident how `appointmentTime` has no use whatsoever. Indeed, at the moment this parameter has the only role of informing the rider about the right time to show up for pick up.

As this is only the simplest interpolation we could come up with, nothing forbids to create more fine-tuned functions to estimate the reimbursement size.

Remark 3.4.6 (Mutually assured loss)**.** The security-aware reader will have noticed how Remark 3.4.4 creates surface for an attack. Indeed, a malicious user may create a fictictious restaurant and create fake `R-Tx` with high `insurance`. When rider emits a `T-Tx` to pick the order up he becomes aware that the `R-Tx` was a scam, and is forced to cancel `T-Tx` effectively 'donating' part of the escrow to the scammer.

This hypothetical attack is avoided by noticing that rider may also choose not to sign a ~~`T-Tx`~~. In this way, rider keeps the escrow locked, losing some money, but the scammer looses `fee` as well. After a number of blocks, fixed at protocol level, have passed, the `R-Tx` and `T-Tx` are automatically forfeited and all escrows are donated to the protocol. Then they can be further used to reimburse rider or any other party via a governance system.

3.5 Pick-up transaction



Pick-up transactions (`P-Tx`) register the fact that a rider has effectively picked up the package to be delivered. In doing so, rider commits to the possibility of entirely losing the escrow defined in Definition 3.3.1.

Definition 3.5.1 (Pick-up transaction)**.** *A* pick-up transaction*, also denoted `P-Tx`, is composed of the following fields:*

`T-Tx`**:** *the hash of a `T-Tx`;*

`nonce`**:** *a positive integer, ensuring that the same transaction is not repeated.*

To be valid, the transaction must satisfy the following conditions:

- *`T-Tx` must be the hash of a valid `T-Tx` transaction;*
- *No other transaction must point to `T-Tx`;*
- *It must be `lastNonce` $\leq$ `nonce`, where `lastNonce` is the nonce of the last transaction signed by `signee`. `lastNonce` is considered to be* 0 *if no such transaction exists;*
- *`signee` must be `T-Tx.signee`.*

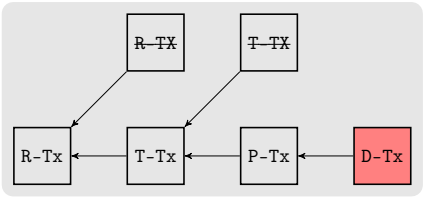
The transaction has the following effects:

- *`baseFee` is released from the `T-Tx.signee` escrow.*

Remark 3.5.2 (A word of caution about handing packages)**.** As a `P-Tx` is signed only by the rider taking charge of the delivery, sender should wait for the `P-Tx` to be confirmed before handling the package to rider, otherwise escrow protection will not be guaranteed.

Remark 3.5.3 (`signee` has no interest in bad behavior)**.** In contrast to Remark 3.5.2, we notice that the `signee` of a `T-Tx` has no interest in signing a `P-Tx` without having taken possession of the package first, as `P-Tx` makes it effectively impossible for rider to get `T-Tx.R-Tx.insurance` back from escrow if the delivery is not completed.

3.6 Delivery transaction



Delivery transactions (`D-Tx`) represent the conclusion of a delivery process. It signals the fact that a given package has been handled to its final recipient.

Definition 3.6.1 (Delivery transaction)**.** *A* delivery transaction*, also denoted* `D-Tx`*, is composed of the following fields:*

`P-Tx`**:** *the hash of a* `P-Tx`*;*

`disputeData`**[optional]:** *a positive integer.*

`nonce`**:** *a positive integer, ensuring that the same transaction is not repeated.*

To be valid, the transaction must satisfy the following conditions:

- `P-Tx` *must be the hash of a valid* `P-Tx` *transaction;*
- *No other transaction must point to* `P-Tx`*;*
- *It must be* $\texttt{lastNonce} \leq \texttt{nonce}$*, where* `lastNonce` *is the nonce of the last transaction signed by* `signee`*.* `lastNonce` *is considered to be* 0 *if no such transaction exists;*
- `signee` *must be either* `P-Tx.T-Tx.R-Tx.from` *or* `P-Tx.T-Tx.R-Tx.to`*;*
- *If* `disputeData` *is present, it must be* $0 < \texttt{disputeData} \leq \texttt{P-Tx.T-Tx.R-Tx.insurance}$*;*
- *If* `disputeData` *is present, the transaction must be also signed by* `P-Tx.signee`*.*

The transaction has the following effects:

- *An amount* `deliveryFee` *(cfr. Definition 3.1.1) from* `P-Tx.T-Tx.R-Tx.signee` *escrow is given to* `signee`*;*
- *The remaining amount* `baseFee` *(cfr. Definition 3.1.1) from* `P-Tx.T-Tx.R-Tx.signee` *escrow is donated to the protocol;*
- *If* `disputeData` *is present:*
  - *An amount* `disputeData` *is taken from* `P-Tx.T-Tx.signee` *escrow and it is given to* `P-Tx.T-Tx.R-Tx.signee`*;*
  - *The remaining amount* $\texttt{P-Tx.T-Tx.R-Tx.insurance} - \texttt{disputeData}$ *from the escrow of* `P-Tx.T-Tx.signee` *is returned to* `P-Tx.T-Tx.signee`*.*
- *If* `disputeData` *is not present we reason as in the above point setting* $\texttt{disputeData} = 0$*.*

Remark 3.6.2 (Both `from` and `to` can sign a `D-Tx`)**.** The reason why we allow for both `from` and `to`, originally defined in `P-Tx.T-Tx.R-Tx` to sign a delivery transaction is to accomodate for failed delivery. Normally, `to`, representing the recipient, would be the one receiving the delivery, after which `to` would sign a delivery transaction to unlock rider's escrow and allowing him to collect the delivery fee. Sometimes though, delivery may fail. In such a circumstance, the best strategy for rider would be going back to sender with the package, giving it back, and receiving escrow and delivery fee anyway (rider has no fault in such a circumstance). To make this into a viable strategy `from`, representing sender, must be able to sign the `D-Tx`.

Remark 3.6.3 (The role of `disputeData`)**.** It may happen that recipient is not satsfied with the delivery service. In this case, a dispute between rider and recipient may arise. These two parties may come to some form of agreement where rider partially reimburses recipient for the throuble caused. The result of this agreement is represented by `disputeData`: after having agreed on a reimbursement amount, one of the two parties prepares a `D-Tx` transaction and signs it. The transaction is privately shared to the other party, that signs it as well. Only then, the transaction is broadcasted, and the agreement is automatically carried out by the protocol.

3.7 Slashing transactions

The attemptive reader may have noted that there may be situations where things are left "pending": For instance, there may be a `T-Tx` that has no ~~`T-TX`~~ nor `P-Tx` pointing to it. In this case, the `T-Tx` is not canceled nor transformed into an actual pick-up. This behavior is intended, for instance in the case of a restaurant making the rider wait longer since the order is not yet prepared.

Maintaining things in this 'undetermined state' gives some slack to the actors involved, and makes the protocol more flexible. Still, there is no utility in wasting money by locking it in escrow forever. For this reason, after a given number of ticks pass after `T-Tx` is mined or validated, the `T-Tx` can be automatically *slashed* if no valid `P-Tx` or ~~`T-TX`~~ points to it. By *slashing*, here we mean that all the money locked in escrow is donated to the protocol. Whoever runs the protocol – that is, miners or validators – has interest in slashing transactions as this increases protocol rewards.

A similar strategy is adopted for `P-Tx` that are not followed by any `D-Tx` for a given number of ticks. In this case the delivery is considered lost, rider pays for it in full and the pending escrow is donated to the protocol for the benefit of the entire community.

Chapter 4

Protocol execution

In this chapter, we outline how transactions are executed. The attemptive reader will probably have grasped already how the protocol works by just reasing the transaction definitions.

In any case, the life-cycle of a delivery can be seen as a 'dependent finite state machine' as depicted in Figure 4.1, where inputs are specified in terms of valid transactions. Here, we are using the notation `input‖effects`: the left hand side denotes the kind of transaction needed to make the finite state machine change its state, whereas the right hand side denotes the effects that this change of state produces. In our case, the effects have been summarized for space reasons, and their full description can be found in chapter 3. The states of the finite state machine are `R` (requested), `T` (taken), `P` (picked-up). The special states `0` and `1` denote the initial and final states, where the delivery lifecycle begins and ends, respectively.

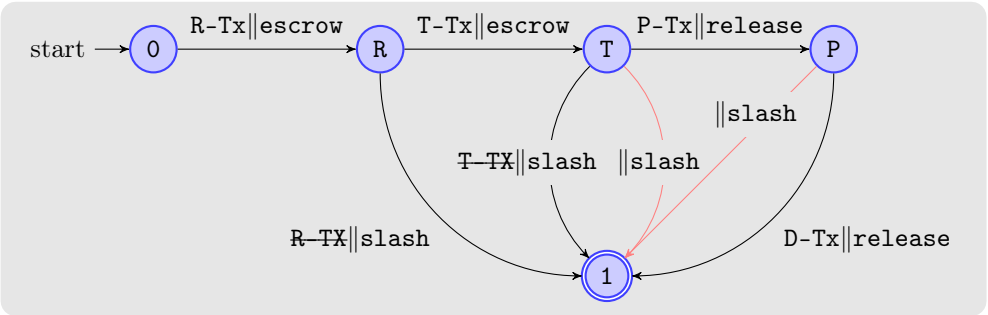


Figure 4.1: Protocol execution as a finite state machine.

Remark* 4.0.1 (Formalising dependent finite state machines)**.** Formally, our finite state machine is dependent, in that 'the state has state' – for instance in `R` we have to keep track of various parameters such as `insurance` and `fee`.

Formally, a dependent finite state machine can be relized by following [7]: Remembering that the execution space of a state machine is nothing more than the free category generated by its graph, dependency can be realized by considering functors from this category to the category **Span** of sets and spans between them. In [7], we explain this process and call it 'guarded Petri nets with side-effects', but it can be readily adapted to the case at hand by relaxing the monoidality requirements.

Performing the Grothendieck construction on this functor we obtain again a free category. In short, our 'dependent finite state machine' can be compiled back to a 'state machine': the only thing we lose is the finiteness of the state space.

Bibliography

[1] A. Back, "Hashcash - a denial of service counter-measure," 2002 (cit. on p. 3).

[2] H. Buhrman, N. Chandran, S. Fehr, R. Gelles, V. Goyal, R. Ostrovsky, and C. Schaffner, "Position-Based Quantum Cryptography: Impossibility and Constructions," *SIAM Journal on Computing*, vol. 43, no. 1, pp. 150–178, Jan. 2014. [Online]. Available: http://arxiv.org/abs/1009.2490 (visited on 03/29/2022) (cit. on p. 7).

[3] Centre. (2018). "Centre Whitepaper," [Online]. Available: https://f.hubspotusercontent30.net/hubfs/9304636/PDF/centre-whitepaper.pdf (cit. on p. 8).

[4] N. Chandran, V. Goyal, R. Moriarty, and R. Ostrovsky, "Position Based Cryptography," in *Advances in Cryptology - CRYPTO 2009*, ser. Lecture Notes in Computer Science, S. Halevi, Ed., vol. 5677, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 391–407. [Online]. Available: http://link.springer.com/10.1007/978-3-642-03356-8_23 (visited on 03/29/2022) (cit. on p. 7).

[5] H. Dong, S. Zhong, S. Xu, J. Tian, and Z. Feng, "The Relationships Between Traffic Enforcement, Personal Norms and Aggressive Driving Behaviors Among Normal e-Bike Riders and Food Delivery e-Bike Riders," *Transport Policy*, vol. 114, pp. 138–146, Dec. 2021. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0967070X21002791 (visited on 03/29/2022) (cit. on p. 4).

[6] Frax Finance. (2022). "FRAX Whitepaper," [Online]. Available: https://docs.frax.finance/overview (cit. on p. 8).

[7] F. Genovese and D. I. Spivak, "A Categorical Semantics for Guarded Petri Nets," in *Graph Transformation*, ser. Lecture Notes in Computer Science, F. Gadducci and T. Kehrer, Eds., vol. 12150, Cham: Springer International Publishing, 2020, pp. 57–74. [Online]. Available: http://arxiv.org/abs/2002.02762 (visited on 12/15/2020) (cit. on p. 20).

[8] M. Hicks, B. Stickle, and J. Harms, "Assessing the Fear of Package Theft," *American Journal of Criminal Justice*, Jan. 4, 2021. [Online]. Available: http://link.springer.com/10.1007/s12103-020-09600-x (visited on 03/29/2022) (cit. on p. 4).

[9] Maker DAO. (2017). "DAI Whitepaper," [Online]. Available: https://makerdao.com/en/whitepaper/ (cit. on p. 8).

[10] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," *www.bitcoin.org*, pp. 1–9, 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf (cit. on p. 3).

[11] B. A. Sparks and V. Browning, "The Impact of Online Reviews on Hotel Booking Intentions and Perception of Trust," *Tourism Management*, vol. 32, no. 6, pp. 1310–1323, Dec. 2011. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0261517711000033 (visited on 03/29/2022) (cit. on p. 4).

[12] Wikipedia Contributors. (2022). "Decimal Degrees," Wikipedia, the free encyclopedia, [Online]. Available: https://en.wikipedia.org/wiki/Decimal_degrees (visited on 03/30/2022) (cit. on p. 10).

[13] ——, (2022). "Last Mile," Wikipedia, the free encyclopedia, [Online]. Available: https://en.wikipedia.org/wiki/Last_mile_(transportation) (visited on 03/29/2022) (cit. on p. 3).

[14] ——, (2022). "Lyft," Wikipedia, the free encyclopedia, [Online]. Available: https://en.wikipedia.org/wiki/Lyft (visited on 03/29/2022) (cit. on p. 4).

[15] ——, (2022). "Uber," Wikipedia, the free encyclopedia, [Online]. Available: https://en.wikipedia.org/wiki/Uber (visited on 03/29/2022) (cit. on p. 4).